Greenfield deployment runbook

The reusable, org-agnostic path from **an empty VPC in a GovCloud landing-zone spoke account** to **a Windows client installed and authenticated end to end** against the Claude apps gateway. Its siblings: `om-runbooks.md` covers **steady state after** this runbook finishes (rotations, updates, alarms, teardown), and `troubleshooting.md` is the symptom-indexed troubleshooting reference — go there when a step here fails. Deep explanations are linked, not duplicated: this document is the spine, every command in order, with the org-prerequisite lead times sequenced first because they, not AWS, set the calendar.

Two hosts (`.claude/rules/offline-build.md`): an **egress host** (internet, no AWS needed) runs the `scripts/mirror/` tools, and the **build/deploy machine** (Docker + AWS service endpoints only, **no internet**) runs everything else against the transferred `mirror/` directory. The split lands in Phase 4; if the two hosts keep separate checkouts, remember `deploy.env` is persisted locally on whichever host ran a script — the build/deploy machine's copy is the one that accumulates `KMS_KEY_ARN`, `IMAGE_URI`, `DBADMIN_IMAGE`, `GRAFANA_IMAGE`, `COLLECTOR_IMAGE`, and `CERTIFICATE_ARN`.

Shell convention: command examples expand variables like `$GATEWAY_FQDN` and `$CLAUDE_VERSION` in **your** shell — the scripts source `deploy.env` internally, but argument expansion does not, so load it first: `set -a; . scripts/deploy.env; set +a`.

Legend: ☐ = do it · ◆ = checkpoint, confirm before moving on.

Phases: **0** org prerequisites → **1** account & workstation prep → **2** certificate → **3** database (01) → **4** mirror + images → **5** gateway (02) → **6** DNS + Zscaler activation → **7** verify + Okta secret → **8** observability (03) + 02 re-run → **9** optional portal (04) → **10** client rollout → **11** end-to-end validation.

Phase 0 — Org prerequisites: send these FIRST

Three request emails go to three different teams, and their turnaround — not the AWS work — dominates the schedule. All three are actionable on day one; send them before touching AWS. Templates (fill placeholders from `scripts/deploy.env`):

- ☐ **Networking** / **PKI** / **Zscaler** — `../requests/networking-request-email.md`. Three asks in one mail: the **enterprise-CA certificate** (CSR signed against `GATEWAY_FQDN` — actionable immediately, no AWS dependency), the **internal DNS CNAME** (the *name* can be reserved now; the *target value* is the `AlbDnsName` stack output that exists only after Phase 5 — tell the DNS team a same-day follow-up carries it), and **Zscaler access**, which has **two independent halves**:
- the **client-side** entry for `GATEWAY_FQDN` (ZPA app segment on TCP 443, or ZIA SSL-inspection exemption + app bypass) — TLS inspection here breaks the client's certificate fingerprint pin, and `verify-gateway.sh` hard-fails if it sees a Zscaler-issued cert; and
- the **server-side egress** ALLOW **plus SSL-inspection exemption** for the **Okta issuer FQDN** on the ZIA location that carries the workload VPC's central egress. This is server-originated traffic with no Zscaler user identity, so default policy both intercepts and blocks it. **Without it the gateway does not boot** — it fails at OIDC discovery with a 403. Do not treat it as optional or as covered by half 1: it is a separate rule on a separate path, and it is the single most common schedule blocker for a new deployment.

- ☐ **Okta administrator** — `../requests/okta-request-email.md`. An OIDC **Web** app (confidential — it must have a client secret) on the **org authorization server**, with **both the Authorization Code AND Refresh Token grants** (without Refresh Token, `offline_access` is silently ignored and every developer is bounced through browser SSO at the session TTL — as short as 1 hour), all needed redirect URIs (`/oauth/callback`, `/grafana/login/generic_oauth`, and `/portal/oauth/callback` if deploying the portal), a **groups claim** configured on the app (the `groups` scope alone is not enough; without the claim, Grafana role mapping and portal access deny everyone, and per-group spend caps cannot resolve), and the groups themselves (`GRAFANA_ADMIN_GROUP`, `ACCESS_GROUP`) with your test user in them.
- ☐ **AD / GPO (or MDM) team** — `../requests/ad-request-email.md`. One machine-policy value (GPP Registry item to `HKLM\SOFTWARE\Policies\ClaudeCode`, recommended) carrying `forceLoginMethod: "gateway"` + `forceLoginGatewayUrl`. Without it the "Cloud gateway" login **does not exist** on any client — it is honored only from a managed source, by Anthropic's anti-phishing design. The `GATEWAY_FQDN` value is static and known now, so this GPO can be built and staged immediately; it only has to be *linked/enabled* by Phase 10.

Dependency map (the chicken-and-egg items, explicit):

Item	Ready when	Blocks
Signed certificate	CA turnaround only	Phase 5 (ALB listener) — actually Phase 2 import
DNS CNAME (name)	reservable now	—
DNS CNAME (target)	**after Phase 5** (`AlbDnsName`)	Phase 6/7 (verify, login)
Zscaler client-side (gateway FQDN)	Zscaler turnaround	Phase 7+ (any client reaching the ALB)
Zscaler server-side (Okta issuer ALLOW + exemption)	Zscaler turnaround	**Phase 5** — gateway boot fails at OIDC discovery (403) without it
Okta app (ID + secret + groups claim)	Okta turnaround	Phase 5 (boot needs a resolvable issuer + client ID), Phase 7 (secret)
GPO managed login setting	AD turnaround	Phase 10 (no gateway login option without it)

☐ **What you need back before Phase 1 ends**: the **App Connector source CIDR(s)** (→ `CLIENT_INGRESS_CIDR` — the ALB SG locks to exactly these), the Okta **client ID** (→ `OKTA_CLIENT_ID`) and **client secret** (held for the `set-*-secret.sh` prompts — never written to `deploy.env`), the issuer (→ `OKTA_ISSUER`), confirmed group names, and the cert validity period + renewal owner.

Phase 1 — Account & workstation prep

Tooling & identity - ☐ AWS credentials for the target account (`aws sts get-caller-identity --region us-gov-west-1`). The deploy identity must be able to create IAM roles, KMS keys, Lambda, RDS, ECS, ELBv2, Secrets Manager, ECR — and **set stack policies** (the deploy scripts call `set-stack-policy`). - ☐ `docker`, `jq`, `openssl`, and AWS CLI v2 on the host.

Bedrock model access - ☐ Enable Claude model access in the account, then confirm the exact GovCloud inference-profile IDs the defaults assume (`us-gov.anthropic.claude-opus-4-8` — un-dated ID, `us-gov.anthropic.claude-sonnet-5` — un-dated, and `us-gov.anthropic.claude-sonnet-4-5-20250929-v1:0` for the small/fast role): `bash aws bedrock list-inference-profiles --region us-gov-west-1 \ --query "inferenceProfileSummaries[?contains(inferenceProfileId,'anthropic')].inferenceProfileId"` If they differ, set `OPUS_BEDROCK_MODEL_ID` / `SONNET_BEDROCK_MODEL_ID` / `HAIKU_BEDROCK_MODEL_ID` in `deploy.env`. Sonnet 5 is the Sonnet-tier default; Sonnet 4.6 was never offered in GovCloud — do not change the defaults without checking this output.

deploy.env - ☐ `cp scripts/deploy.env.example scripts/deploy.env` and fill (the example's comments are the authoritative per-variable documentation): `VPC_ID`, `VPC_CIDR`, `PRIVATE_SUBNET_IDS` (≥2 AZs), `CLIENT_INGRESS_CIDR` (**narrow it** to the App Connector / VPN CIDRs from Phase 0 — `10.0.0.0/8` is an auditor finding and misses on-prem connectors in 172.16/12), `GATEWAY_FQDN`, `OKTA_ISSUER` (must start `https://`), `OKTA_CLIENT_ID`, `ALLOWED_EMAIL_DOMAINS`, `GRAFANA_OKTA_CLIENT_ID` (= `OKTA_CLIENT_ID` if reusing the app), `GRAFANA_ADMIN_GROUP`. - ☐ **Landing-zone spoke decision** (this runbook's target profile: no NAT, TGW to central egress): set `CREATE_SUPPORTING_ENDPOINTS="true"`, `PRIVATE_ROUTE_TABLE_IDS="rtb-..."` (free S3 gateway endpoint — required for image pulls without NAT), keep `CREATE_BEDROCK_ENDPOINT="true"`, and plan `CREATE_AMP_ENDPOINT="true"` for Phase 8. **Exception**: any service the landing zone already centralizes via shared endpoints/PHZs must be skipped locally or the stack fails on a conflicting DNS name — check with `aws route53 list-hosted-zones-by-vpc` and use the per-service `CREATE_*_ENDPOINT` switches (see the README "VPC endpoints" section and the `deploy.env.example` comments). If the landing zone mandates a central proxy for internet egress, set `HTTPS_PROXY_URL`. - ☐ Leave `IMAGE_URI`, `DBADMIN_IMAGE`, `GRAFANA_IMAGE`, `COLLECTOR_IMAGE`, `PORTAL_IMAGE`, `CERTIFICATE_ARN`, `KMS_KEY_ARN`, and the three `OBSERVABILITY_*` vars **empty** — the scripts persist them as they run; there are no copy-paste steps.

GPG decision for the release mirror (it fails closed) - ☐ Either put Anthropic's release-signing public key on the host and `export ANTHROPIC_GPG_KEY=/path/to/key`, **or** deliberately accept TLS-only trust with `export ALLOW_UNVERIFIED_MANIFEST=1`. Without one of these, the very first mirror step in Phase 4 stops. Record the choice — it is a named, auditable override, not a default.

Endpoint-policy pre-check (no-NAT profile only) - ☐ Confirm the `logs` endpoint supports policies in-region (the `ecs` endpoint deliberately carries none for this reason): `bash aws ec2 describe-vpc-endpoint-services --region us-gov-west-1 \ --service-names com.amazonaws.us-gov-west-1.logs \ --query 'ServiceDetails[].VpcEndpointPolicySupported'`

Okta groups pre-check - ☐ Confirm groups actually come back **from a token**, not from metadata: use the Okta app's token preview (or a real login) for a user in the admin group and check that a `groups` array is present. Discovery metadata never lists the claim, so it cannot be verified there — and the `groups` *scope* without a groups *claim* is the single most common cause of "the portal and Grafana deny everyone".

Re-deploying into an account that has run this deployment before - ☐ Every log group carries `DeletionPolicy: Retain` and a fixed name, and the templates pre-create three groups the services would otherwise auto-create — so a re-create collides with whatever a previous deployment (or the services themselves) left behind. Export anything you still need, then delete before deploying: `bash for g in "/aws/rds/instance/${NAME_PREFIX}-store/postgresql" \ "/aws/lambda/${NAME_PREFIX}-db-bootstrap" \ "/aws/lambda/${NAME_PREFIX}-db-rotation"; do aws logs delete-log-group --region "$AWS_REGION" --log-group-name "$g" || true done` The same applies to the retained `/ecs/*` and `/claude/*` groups before a full re-create from scratch.

Phase 2 — Certificate

```
./scripts/import-enterprise-cert.sh csr "$GATEWAY_FQDN"     # EC P-256 key + CSR (append rsa2048 if the CA requires RSA)
#   → hand the CSR to the enterprise CA (serverAuth EKU); collect leaf + chain
./scripts/import-enterprise-cert.sh import "$GATEWAY_FQDN" leaf.pem "$GATEWAY_FQDN.key.pem" chain.pem
```

Prints `CertificateArn:` (persisted as `CERTIFICATE_ARN`) and the **SHA-256 fingerprint — save it**; it is what developers confirm at first login (Phase 10) and must be published before rollout. Imported ACM certs do **not** auto-renew; the stack alarms at `CERT_EXPIRY_ALARM_DAYS` (wire `ALARM_SNS_TOPIC_ARN`), and renewal is `om-runbooks.md` runbook 1.

Phase 3 — Database stack (01) — FIRST

01 comes first because it creates the KMS CMK and persists `KMS_KEY_ARN`, so the ECR repos created in Phase 4 are born CMK-encrypted (ECR encryption is creation-time-only). The RDS storage key is likewise a **day-one decision** — changing it later is a teardown + restore, not an update.

```
./scripts/deploy-database.sh
```

Stack `CREATE_COMPLETE` (Multi-AZ RDS takes ~10–15 min); the `KmsKeyArnResolved` output; the "Locking the database against replacement/deletion (stack policy)" line; `KMS_KEY_ARN` now in `deploy.env`. If it fails with "version 16.x does not exist", the template's default minor is not offered in this region — pin an available one via `DB_ENGINE_VERSION` and re-run:

```
aws rds describe-db-engine-versions --engine postgres --region "$AWS_REGION" \
  --query "DBEngineVersions[?starts_with(EngineVersion,'16.')].EngineVersion" --output text
```

Phase 4 — Mirror the release + build and push all images

Two hosts (`.claude/rules/offline-build.md`): step 4a runs on the **egress host** (internet, no AWS needed); steps 4b and 4f need **both** upstream-registry reach and AWS creds — see their notes; steps 4c–4e and 4g run on the **build machine** (Docker + AWS only, no internet) after you copy `mirror/` over. The GPG decision from Phase 1 gates the release mirror. Needs `KMS_KEY_ARN` from Phase 3.

```
# 4a. EGRESS HOST — verify + stage every external artifact into mirror/
./scripts/mirror/mirror-claude-release.sh "$CLAUDE_VERSION"   # verifies sha256 + (GPG) manifest; fails closed
./scripts/mirror/mirror-grafana-plugin.sh                     # AMP datasource plugin, sha256-pinned (grafana-plugin.pin)
./scripts/mirror/mirror-rds-ca-bundle.sh                      # RDS CA trust bundle (baked into gateway + db-admin images)
# ---- copy the mirror/ directory to the build machine ----
```

```
# 4b. Base images — mirror all four into your ECR, digest-pinned. Needs BOTH
#     upstream-registry reach (Docker Hub + public.ecr.aws) and AWS creds:
#     run it wherever both are available (the egress host with AWS creds, or
#     the build machine if your landing zone lets it reach the upstream
#     registries). Persists GATEWAY/LAMBDA/GRAFANA/PORTAL_BASE_IMAGE, which
#     the builds below consume — the offline build machine cannot pull the
#     upstream defaults. set_env_var writes only the LOCAL deploy.env: when
#     this runs on a different host than the builds, copy the four persisted
#     *_BASE_IMAGE lines into the build machine's scripts/deploy.env (the
#     script prints a reminder).
./scripts/mirror/mirror-base-images.sh

# 4c. BUILD MACHINE — gateway image (stages claude from mirror/, re-verified
#     against the mirror's CHECKSUMS.txt)
./scripts/build-and-push-image.sh                              # persists IMAGE_URI

# 4d. DB-admin Lambda image (bootstrap + rotation)
./scripts/build-and-push-dbadmin.sh                            # persists DBADMIN_IMAGE

# 4e. Grafana image (bakes in the mirrored AMP plugin, re-verified
#     against scripts/mirror/grafana-plugin.pin)
./scripts/build-and-push-grafana.sh                            # persists GRAFANA_IMAGE

# 4f. ADOT collector — mirror the pinned upstream image into ECR. Needs BOTH
#     public.ecr.aws reach and AWS creds: run it wherever both are available
#     (the egress host with AWS creds, or the build machine if your landing
#     zone lets it reach public.ecr.aws).
ADOT_VERSION=v0.49.0 ./scripts/mirror/mirror-collector.sh      # persists digest-pinned COLLECTOR_IMAGE

# 4g. Download-portal image (only if deploying Phase 9)
./scripts/build-and-push-portal.sh                             # persists PORTAL_IMAGE
```

`grep -E 'IMAGE_URI|DBADMIN_IMAGE|GRAFANA_IMAGE|COLLECTOR_IMAGE|BASE_IMAGE' scripts/deploy.env` — all set by the scripts, none by hand. The mirror output also contains `claude.exe` + `claude` + `CHECKSUMS.txt` for the client rollout (Phase 10; the portal publish needs both platform binaries) — stage `mirror/$CLAUDE_VERSION/` on the internal file share now. The ADOT version defaults to the release this repo pins (`ADOT_VERSION` in `scripts/mirror/mirror-collector.sh`); override it only deliberately. Base images come from step 4b's digest-pinned ECR copies (`GATEWAY_BASE_IMAGE`, `LAMBDA_BASE_IMAGE`, `GRAFANA_BASE_IMAGE`, `PORTAL_BASE_IMAGE`) — the build machine cannot reach Docker Hub or `public.ecr.aws`, and the upstream defaults exist for dev convenience only; the builds need no package-repo access at all (README, "Controlled-network image builds").

Phase 5 — Gateway stack (02)

Gate: the server-side Zscaler exemption for the Okta issuer (Phase 0, half 2) must be live — without it the gateway boot fails at OIDC discovery with a 403 and the service never goes healthy. Probe first from an in-VPC host:

```
curl -sv "https://<OKTA_ISSUER_HOST>/.well-known/openid-configuration"   # expect 200 JSON
```

(A Zscaler HTML block page = policy ALLOW missing; a TLS error = the inspection exemption missing — interim fallback is `EXTRA_CA_CERT_PATH`, see `../requests/networking-request-email.md` §3.)

```
./scripts/deploy-gateway.sh
```

Creates the ALB, ECS gateway, IAM, secrets, VPC endpoints, the DB-bootstrap custom resource, the app-secret rotation schedule, and the spend-cap `admin:` block + admin keys. Watch three live-only validations:

- **DB bootstrap** — `Custom::DbAppUserBootstrap` reaches `CREATE_COMPLETE` (on failure it times out in ~5 min; tail `/aws/lambda/<prefix>-db-bootstrap`).
- **Target-group health** — targets go `healthy` on the **HTTPS** health check:

```
aws ecs wait services-stable --region "$AWS_REGION" --cluster "${NAME_PREFIX}-cluster" --services "${NAME_PREFIX}-gateway"
aws elbv2 describe-target-health --region "$AWS_REGION" --target-group-arn <tg-arn>
```

- **First app-secret rotation** (fires automatically, async — the stack is green regardless, so verify, don't assume):

```
aws secretsmanager get-secret-value --region "$AWS_REGION" \
  --secret-id "${NAME_PREFIX}/db-app-user" --query SecretString --output text \
  | jq -r .username        # expect gateway_app_clone after the first rotation
```

If it did not flip, tail `/aws/lambda/<prefix>-db-rotation` and check the `<prefix>-db-rotation-errors` alarm.

- The "Locking the ALB against replacement/deletion" stack-policy line ran.

Note: `deploy-gateway.sh` runs with `--disable-rollback` by default — a failed deploy keeps its healthy resources, so you fix the cause and re-run and the deploy continues from where it stopped (`troubleshooting.md` for recovery patterns).

Phase 6 — DNS + Zscaler activation

- ☐ Send the DNS team the CNAME **target** (the same-day follow-up promised in Phase 0): `GATEWAY_FQDN CNAME <AlbDnsName>` — get it with `./scripts/stack-outputs.sh`. The ELB name is a public record returning private IPs; no split-horizon zone needed, but confirm the resolver does not strip RFC1918 answers (DNS-rebinding protection — asked in the networking email).
- ☐ Confirm the **client-side** Zscaler entry is active (ZPA app segment or ZIA exemption + bypass for `GATEWAY_FQDN`), and that App Connectors can resolve the corporate CNAME (README, "ZPA & landing-zone prerequisites").

Phase 7 — Verify the gateway + set the Okta secret

```
./scripts/verify-gateway.sh
```

DNS (private A records only, no AAAA), TLS chain + fingerprint (cross-checked against ACM), OAuth endpoints answering. Behind ZPA the laptop sees synthetic CGNAT answers — run the DNS assertions from an App Connector's resolution context; the script says so when it detects them. It hard-fails on a Zscaler-issued cert (client-side bypass not active).

- ☐ Set the gateway's Okta client secret (hidden prompt; rolls the service): `./scripts/set-okta-secret.sh`

Phase 8 — Observability (03) + Grafana secret + 02 re-run

Optional stack, strongly recommended (spend visibility). Order within the phase is load-bearing: 03 emits the AMP parameters that the 02 **re-run** consumes to attach the ADOT collector as a **localhost sidecar** in the gateway task — there is no standalone collector service.

```
./scripts/deploy-observability.sh          # AMP + Grafana; persists OBSERVABILITY_AMP_ENDPOINT / _WORKSPACE_ARN / _ACTIVITY_LOG_GROUP
./scripts/set-grafana-oidc-secret.sh       # paste the (same or dedicated) client secret; rolls Grafana
# set TELEMETRY_FAIL_CLOSED="false" in scripts/deploy.env (fail-OPEN first — see below),
# then re-run to attach the sidecar. An env prefix on the command does NOT work:
# common.sh sources deploy.env after it and the file's value wins.
./scripts/deploy-gateway.sh
```

03 `CREATE_COMPLETE`; the `GrafanaOidcRedirectUri` output matches the URI registered in Okta; the three `OBSERVABILITY_*` vars persisted.

After the 02 re-run, the gateway task runs **two containers** (`aws ecs describe-tasks ... --query 'tasks[].containers[].name'` shows the gateway plus `otel-collector`), and collector log streams appear under the `otel` prefix.

- ☐ **First-boot posture, then fail closed.** Deploy the first telemetry-enabled re-run with `TELEMETRY_FAIL_CLOSED="false"` set **in** `scripts/deploy.env` (the example ships it `"true"`; edit the file — a `VAR=... ./scripts/...` command prefix is silently overridden when `common.sh` sources the file): with fail-closed on, the gateway waits on the collector reporting HEALTHY, and with `MinimumHealthyPercent: 100` and no deployment circuit breaker a misconfigured collector health check hangs the rollout indefinitely at `services-stable` with only a generic "task not healthy" — the highest-risk step on this path. Once the `otel-collector` container reports HEALTHY, flip `TELEMETRY_FAIL_CLOSED="true"` in `deploy.env` and re-run `./scripts/deploy-gateway.sh` — the default, SSP-recorded AU-5 posture, where a failed collector stops the task rather than serving unmonitored traffic, with the `${NAME_PREFIX}-missing-telemetry` alarm as the end-to-end backstop.
- ☐ The `missing-telemetry` alarm legitimately fires between the 03 deploy and the telemetry-enabled re-run — it settles to OK once the sidecar's self-metrics heartbeat lands.

Phase 9 — Optional: installer download portal (04)

Independent of 03 — any time after Phase 5. Shares the ALB / FQDN / cert / Zscaler entry (path-based at `/portal`): **no new DNS or Zscaler request**, but it reaches the Okta issuer over the same server-side egress exemption the gateway needs. Prereqs: `PORTAL_IMAGE` (Phase 4g), the `/portal/oauth/callback` redirect URI, the **groups claim** on the Okta app (without it the portal denies everyone), and `ACCESS_GROUP` populated.

```
./scripts/deploy-download-portal.sh                    # stack + CMK artifacts bucket (persists PORTAL_ARTIFACTS_BUCKET)
./scripts/publish-portal-release.sh "$CLAUDE_VERSION"  # re-verifies claude.exe + claude against the manifest, uploads
./scripts/set-portal-oidc-secret.sh                    # paste the portal client secret; rolls the service
```

Target group healthy on the HTTPS `/portal/healthz` check; `PortalOidcRedirectUri` matches Okta. The publish step re-verifies both platform binaries against the release manifest SHA-256 before upload — and must have run before the portal serves downloads, or a Linux download aborts mid-stream against a bucket with no `releases/<version>/claude`. End-to-end validation items are in Phase 11.

Phase 10 — Client rollout (Windows fleet)

The model (full detail: `client-config.md`): **no-admin binary install + one required admin-delivered managed setting for login + server-side policy push**. This phase is the fleet path; the single-laptop bring-up variant is the box below it.

- ☐ **Managed login setting via GPO/MDM** — confirm the Phase-0 GPO (`../requests/ad-request-email.md`) is now linked to the developer OU and applied (`gpupdate /force`; verify with `/status` inside `claude` — the managed source must be listed). Machine scope, Registry mechanism recommended. Without it there is **no gateway login option on any client**. Note the GPO's `forceRemoteSettingsRefresh: true` makes the CLI exit if it

cannot fetch the gateway's managed settings — the gateway must be serving (Phase 7 green) before this lands on machines, or developers get a non-starting CLI.

> **Single laptop instead of a fleet.** Before the GPO exists, seed the same managed value once with an **elevated** PowerShell, then run `claude` non-elevated (the binary install itself stays no-admin): `powershell New-Item -Path 'HKLM:\SOFTWARE\Policies\ClaudeCode' -Force | Out-Null Set-ItemProperty -Path 'HKLM:\SOFTWARE\Policies\ClaudeCode' -Name Settings ` -Value '{"forceLoginMethod":"gateway","forceLoginGatewayUrl":"https://<FQDN>"}'` This bring-up seed deliberately **omits** `forceRemoteSettingsRefresh` (which the real GPO carries): that key makes the CLI exit if it cannot fetch the gateway's managed settings — the posture you want in production, but it leaves you with no working CLI to debug with while the gateway is still coming up. Once login succeeds, add `"forceRemoteSettingsRefresh":true` and re-test so the laptop matches the fleet; the `/model` check in Phase 11 is what confirms the push landed. On Linux the equivalent is the root-owned `/etc/claude-code/managed-settings.json` (`client-config.md` §8.5). - ☐ **Publish the certificate fingerprint** (from Phase 2) through a channel developers trust — they confirm it at first connect (trust-on-first-use pin). Ensure the enterprise root CA is in the Windows cert store (normally already there by GPO) — chain validation happens *before* the pin. - ☐ **Distribute the installer** — either path: - **Portal** (Phase 9): developers browse `https://<GATEWAY_FQDN>/portal` → Okta SSO → pick Cost Center, one of its Teams, and a Platform (Windows or Linux x64) → ZIP with a pre-baked wrapper — `install.cmd` (`-GatewayUrl` / `-Sha256` / `-Team` / `-CostCenter` / `-DisableUpdates`) on Windows, `install.sh` (same options) on Linux. - **Direct** (file share): stage `claude.exe` + `CHECKSUMS.txt` from the Phase-4 mirror on the share (over ZPA the share needs its **own** app segment, TCP 445), then per laptop, **no admin**: `powershell powershell -ExecutionPolicy Bypass -File .\client\Install-ClaudeCode.ps1 ` -BinaryPath \\<fileserver>\<share>\claude\$env:CLAUDE_VERSION\claude.exe ` -Sha256 <platforms.win32-x64.checksum from manifest.json> ` -GatewayUrl https://<GATEWAY_FQDN> ` -Team <team> -CostCenter <cc> -DisableUpdates` Deploy in **user** context — the installer refuses SYSTEM runs. Machines provisioned by an *older* installer need stale managed settings cleared first (`client-config.md` §8.4). - ☐ **One-time login**: new terminal → `claude` → the login screen is locked to gateway with the URL pre-filled (no picker, no typing) → press Enter → one-time **Okta SSO** in the browser (+MFA) → confirm the fingerprint against the published value → a prompt returns a Bedrock completion. The token persists with refresh — if developers are bounced through SSO hourly, the Okta app is missing the Refresh Token grant (Phase 0). Tell developers: sign out with `claude auth logout`, **never** `/logout` — `/logout` strands the client behind an unreachable-hosts preflight (recovery: `om-runbooks.md` runbook 12). - ☐ **Clean-profile first run**: verify a first-ever run on a profile that has *never* run `claude`, not just a re-login. With `hasCompletedOnboarding` unset, a **connectivity preflight** runs ahead of the login screen: it requires HTTP 200 from both `api.anthropic.com` and `platform.claude.com`, which a gateway-only egress path does not provide, and the CLI exits with *"Unable to connect to Anthropic services"*. Both installers therefore seed `hasCompletedOnboarding: true` at install time (creating `.claude.json` when the profile has none) — this check verifies the seeding worked: after the install, the state file must contain the flag, and `claude` must start to the login screen, not the preflight error. If it reproduces anyway, the deployed installer predates the seeding — re-run `publish-portal-release.sh` so the portal serves the current `client/` installers. Per-developer recovery and the matching `/logout` trap are `om-runbooks.md` runbook 12.

Phase 11 — End-to-end validation checklist

The deployment is done when every box below is checked — until then it is not production-ready. Where a check fails, start at `troubleshooting.md` and the alarm runbook (`om-runbooks.md` runbook 9).

- ☐ **Gateway health**: `./scripts/verify-gateway.sh` all green; both targets healthy; task authenticates to Postgres as `gateway_app*`, never the master user (check `/ecs/<prefix>` logs for the clean DB connect).
- ☐ **Developer login round-trip**: locked gateway login → Okta SSO → fingerprint match → Bedrock completion.
- ☐ **Model picker constrained**: in a logged-in session, `/model` lists **only** `OPUS_MODEL_ID`, `SONNET_MODEL_ID` and `HAIKU_MODEL_ID` (three entries) — not Claude Code's built-in menu. Send a prompt on each; confirm background/small-fast tasks also succeed (the Haiku-override push resolves them to `HAIKU_MODEL_ID`). If the built-in menu appears, the running gateway image predates the `GATEWAY_MANAGED_B64` stanza and ignores the env var silently — rebuild with a bumped tag (`troubleshooting.md`).
- ☐ **Telemetry flowing**: after a few sessions, `claude_code_*` series appear in AMP and the Grafana cost/token panels populate. Confirm `otelcol_exporter_prometheusremotewrite_failed_translations` stays flat — a climbing counter with client activity is the delta-temporality trap, which the pushed cumulative-temporality env var fixes (verify it took on this deployment). First stop when metrics are missing: `./scripts/diagnostics/diagnose-telemetry.sh` (walks the client → gateway → sidecar → AMP chain); `./scripts/diagnostics/amp-query.py` queries AMP directly.
- ☐ **Grafana Okta SSO**: `https://<GATEWAY_FQDN>/grafana` → "Sign in with Okta" → a `GRAFANA_ADMIN_GROUP` member lands as Admin and the usage dashboard renders; a user in no mapped group is denied (strict mapping).
- ☐ **Spend-cap smoke test**: set a low cap on the test user, confirm the 429 with `SPEND_BLOCKED_MESSAGE` once exceeded, then clear it (needs `GATEWAY_CA_BUNDLE` set for the script's TLS): `bash ./scripts/set-spend-limit.sh --scope user --id <okta-sub|email> --amount 1 --period daily ./scripts/set-spend-limit.sh --list ./scripts/set-spend-limit.sh --scope user --id <okta-sub|email> --clear` Note the availability trade the stack enables (`enforcement.fail_closed_on_error`): a spend-store outage halts all inference — recovery is `cost-controls.md` §5.
- ☐ **pgaudit**: the `/aws/rds/instance/${NAME_PREFIX}-store/postgresql` log group receives DDL/connection events.
- ☐ **Rotation proof**: `db-app-user` AWSCURRENT username flipped to `gateway_app_clone` (Phase 5 checkpoint), or flips on a manual `aws secretsmanager rotate-secret --secret-id "${NAME_PREFIX}/db-app-user"`, and the gateway service rolled afterward.
- ☐ **Alarms wired**: `ALARM_SNS_TOPIC_ARN` set and subscribed; `missing-telemetry` alarm OK (its OK→ALARM→OK cycle is cheap to test — stop the sidecar); cert-expiry and db-rotation-errors alarms exist.
- ☐ *(if Phase 9)* **Portal**: Okta login + a real download of each platform ZIP works end to end; a non-`ACCESS_GROUP` user is denied **and** the denial plus successful downloads land in the `/claude/<prefix>/portal-audit` log group.
- ☐ *(if enabled)* **Activity archive**: with `FORWARD_ACTIVITY_LOGS=true`, events land in the CloudWatch window and the S3 archive. Treat the stream as highly sensitive.

One-page command summary (landing-zone spoke, happy path)

```
# Phase 0: send the three request emails; wait for cert, Okta app, Zscaler halves, GPO
# Phase 1: fill scripts/deploy.env; enable Bedrock model access;
#          export ANTHROPIC_GPG_KEY=... (or ALLOW_UNVERIFIED_MANIFEST=1)
./scripts/import-enterprise-cert.sh csr "$GATEWAY_FQDN"
./scripts/import-enterprise-cert.sh import "$GATEWAY_FQDN" leaf.pem "$GATEWAY_FQDN.key.pem" chain.pem
./scripts/deploy-database.sh
# -- egress host --
./scripts/mirror/mirror-claude-release.sh "$CLAUDE_VERSION"
./scripts/mirror/mirror-grafana-plugin.sh
./scripts/mirror/mirror-rds-ca-bundle.sh
# -- dual-reach host (upstream registries + AWS creds; see Phases 4b/4f) --
./scripts/mirror/mirror-base-images.sh                    # digest-pins *_BASE_IMAGE
ADOT_VERSION=v0.49.0 ./scripts/mirror/mirror-collector.sh   # digest-pins COLLECTOR_IMAGE
# -- copy mirror/ (and, if hosts differ, the persisted *_BASE_IMAGE +
#    COLLECTOR_IMAGE deploy.env lines) to the build machine; everything below runs there --
./scripts/build-and-push-image.sh
./scripts/build-and-push-dbadmin.sh
./scripts/build-and-push-grafana.sh
./scripts/deploy-gateway.sh                     # gate: Okta-issuer egress exemption live
#   ... DNS CNAME target to the DNS team; confirm client-side Zscaler entry ...
./scripts/verify-gateway.sh
./scripts/set-okta-secret.sh
./scripts/deploy-observability.sh
./scripts/set-grafana-oidc-secret.sh
#   ... set TELEMETRY_FAIL_CLOSED="false" in deploy.env (env prefix won't stick) ...
./scripts/deploy-gateway.sh                     # attach sidecar fail-open
#   ... collector HEALTHY? flip TELEMETRY_FAIL_CLOSED="true" in deploy.env ...
./scripts/deploy-gateway.sh
#   ... optional portal ...
./scripts/build-and-push-portal.sh
./scripts/deploy-download-portal.sh
./scripts/publish-portal-release.sh "$CLAUDE_VERSION"
./scripts/set-portal-oidc-secret.sh
#   ... Phase 10 client rollout; Phase 11 validation checklist ...
```

Teardown (should you need to start over) is the reverse — 04 and 03, then 02, then 01: `om-runbooks.md` runbook 13, which also covers the re-create caveats (retained log groups, the Secrets Manager recovery window, lingering Lambda ENIs).

Appendix — lab shortcut: a self-signed ALB certificate

For a throwaway or lab environment where the enterprise CA is not yet in the loop, a self-signed certificate for the FQDN exercises the real trust flow: Claude Code validates the chain first, then pins the fingerprint, so the full TLS + fingerprint-pin path behaves exactly as it does in production. **Never publish a lab fingerprint as production-trusted.**

```
set -a; . scripts/deploy.env; set +a
FQDN="$GATEWAY_FQDN"

# 1. Self-signed cert for the FQDN (EC P-256, serverAuth, SAN = FQDN)
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 -nodes \
  -keyout "${FQDN}.key.pem" -out "${FQDN}.crt.pem" -days 90 \
  -subj "/CN=${FQDN}" \
  -addext "subjectAltName=DNS:${FQDN}" \
  -addext "keyUsage=digitalSignature" \
  -addext "extendedKeyUsage=serverAuth"

# 2. Import into ACM (no chain for a self-signed cert) and record the ARN
ARN=$(aws acm import-certificate --region "$AWS_REGION" \
  --certificate "fileb://${FQDN}.crt.pem" \
  --private-key  "fileb://${FQDN}.key.pem" \
  --query CertificateArn --output text)
( source scripts/common.sh; set_env_var CERTIFICATE_ARN "$ARN" )

# 3. The fingerprint developers compare at the /login prompt
openssl x509 -in "${FQDN}.crt.pem" -noout -fingerprint -sha256
```

Then trust it on the client, or TLS fails before the fingerprint prompt is ever drawn. Either import `${FQDN}.crt.pem` into the Windows cert store (`Import-Certificate -CertStoreLocation Cert:\CurrentUser\Root`, no admin needed), or pass `-ExtraCaCertPath` to the installer (writes `NODE_EXTRA_CA_CERTS`) — the same mechanism used for a real enterprise CA, so it is the more useful of the two to exercise. A self-signed leaf is its own trust anchor, so the leaf itself is what goes into the store or PEM; there is no separate CA to import. Do **not** reach for `NODE_TLS_REJECT_UNAUTHORIZED=0`.